

LLM-Based Analysis of User Comments for HarmonyOS Applications

Lin Wenqi Hu Haozhan Wang Zhaohe

June 1, 2026

1 Introduction

1.1 Motivation

HarmonyOS, developed by Huawei, is a distributed operating system designed for a wide range of devices including smartphones, tablets, wearables, and IoT equipment. Since its official release, the HarmonyOS ecosystem has expanded rapidly, with the Huawei AppGallery now hosting hundreds of thousands of native HarmonyOS applications [1].

As the ecosystem matures, user comments on the AppGallery have become a critical channel for developers to understand user satisfaction, identify bugs, and prioritize feature improvements. Unlike structured bug reports or crash logs, app store reviews are unstructured, multilingual, and often mix multiple concerns within a single comment. For example, a single review might praise the UI design while simultaneously reporting a login failure on a specific device model.

Existing approaches to app review analysis largely target the Android (Google Play) and iOS (App Store) ecosystems [3, 4]. However, the HarmonyOS platform introduces unique challenges:

- **Closed ecosystem tooling.** Unlike Android’s adb-based automation, HarmonyOS devices use the proprietary HDC (HarmonyOS Device Connector) protocol with limited tooling support.
- **Diverse device landscape.** HarmonyOS runs on a heterogeneous mix of hardware, making device-specific compatibility issues more prevalent.
- **Rapid API evolution.** The platform is evolving quickly, and compatibility problems between API versions often surface in user reviews.

These factors motivate the development of a specialized pipeline tailored to the HarmonyOS application ecosystem.

1.2 Contributions

The main contributions of this work are:

1. A **UI-automation crawler** that reliably scrapes user comments from the Huawei AppGallery on HarmonyOS devices using the `hmdriver2` library and HDC protocol.
2. An **LLM-based classification pipeline** that performs multi-dimensional sentiment analysis on user comments and extracts concrete, actionable suggestions for developers.
3. A **two-stage clustering workflow** that groups semantically similar issues within each dimension-sentiment bucket and merges cross-batch redundancies, with label standardization.
4. An **interactive web dashboard** that presents aggregated results—sentiment distribution, dimension breakdown, topic clusters, and raw comments—in an intuitive visual format.
5. A **curated application dataset** spanning multiple categories (social, e-commerce, finance, entertainment, etc.) on the HarmonyOS AppGallery, with over 10,000 comments collected, serving as a testbed for the analysis pipeline.

2 Background and Related Work

2.1 The HarmonyOS Application Ecosystem

HarmonyOS is a microkernel-based, distributed operating system first released by Huawei in 2019. Unlike Android and iOS, HarmonyOS is designed with a *device-agnostic* philosophy: a single application binary can adapt its UI and behavior across phones, tablets, foldables, smartwatches, and in-vehicle displays. The platform’s native application framework, ArkUI, uses a declarative programming model with the ArkTS language (a TypeScript superset).

The Huawei AppGallery serves as the official distribution channel for HarmonyOS applications. As of 2025, the AppGallery hosts over 500,000 native HarmonyOS applications [2]. Each application page includes structured metadata (title, subtitle, category) and user-generated reviews consisting of a username, star rating (1–5), textual content, date, and device information.

2.2 Mobile App Review Analysis

Analyzing user reviews to inform software maintenance and evolution is an active research area. Early work [3] conducted an empirical study on the Apple App Store, finding that a significant portion of reviews contain actionable information for developers but are buried among uninformative ratings. A subsequent survey [4] of app review analysis techniques found that NLP-based classification of reviews by topic (bug reports, feature requests, user experience) consistently provides value to development teams.

More recently, the emergence of large language models (LLMs) has opened new possibilities for review analysis. Recent studies [5] demonstrated that GPT-4-level models can achieve high accuracy in extracting fine-grained issue types and sentiment from app reviews without task-specific training data. Our work builds on this insight by applying LLM-based analysis specifically to the HarmonyOS ecosystem.

2.3 Mobile UI Automation and Testing

Automated interaction with mobile applications is essential for both testing and data collection. On Android, tools like UIAutomator [8] and Appium [9] provide cross-platform UI automation. On HarmonyOS, the equivalent is HDC (HarmonyOS Device Connector), a command-line tool analogous to `adb`, paired with the `hmdriver2` library [7]. These tools enable programmatic UI inspection, element location via XPath, gesture simulation (tap, swipe), and screenshot capture on connected HarmonyOS devices.

In our pipeline, `hmdriver2` serves as the foundation for the comment scraper, providing the ability to navigate the AppGallery UI and extract comment data that is not accessible through any public API.

3 System Architecture

The AGCOMMENTSPY pipeline consists of four main stages, as illustrated in Figure 1:

1. **Comment Scraping** — Automated UI navigation and text extraction from the AppGallery.
2. **LLM Classification** — Sentiment and dimension classification with actionable suggestion extraction.
3. **Topic Clustering** — Semantic grouping of classified comments into coherent issue clusters.
4. **Visualization Dashboard** — Web-based presentation of aggregated results.

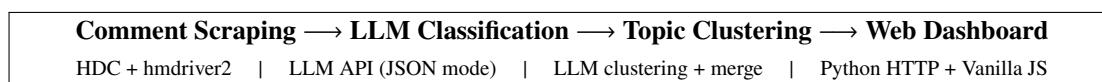


Figure 1: High-level architecture of AGCOMMENTSPY.

3.1 Comment Scraping

The scraping module interacts with a physical HarmonyOS device via HDC. It uses the `hmdriver2` library to:

- Launch the AppGallery application and navigate to a target app’s detail page using a deep-link URL of the form `https://appgallery.huawei.com/app/detail?id=<package_name>`.
- Scroll through the comment section with configurable swipe gestures (default: max 500 swipes) to load historical reviews.
- Parse each comment’s UI hierarchy (dumped as JSON-then-XML) to extract the username, star rating, textual content, date, location, and device model.
- Expand truncated long comments by programmatically clicking “expand” buttons detected via XPath.
- Save raw comments as JSON files keyed by package name for downstream processing.

Listing 1 shows a simplified version of the scraping loop.

Listing 1: Simplified comment scraping loop (from `comments.py`).

```

1 def scrape_comments(driver, max_swipes=50, wait=2.0):
2     all_comments = []
3     for i in range(max_swipes):
4         driver.swipe(...)          # Scroll down
5         layout = driver.dump_hierarchy()
6         xml = json2xml(layout)
7         for node in xml.xpath(COMMENT_ITEMS):
8             comment = parse_comment(node)
9             all_comments.append(comment)
10    return all_comments

```

3.2 LLM-Based Comment Classification

Once raw comments are collected, the classification module sends batches of comments (default: 10 per batch) to an LLM API (OpenAI-compatible). Each comment is analyzed along two dimensions:

Sentiment — Each extracted issue is labeled as *Positive*, *Negative*, or *Neutral*.

Core Dimension — Each issue is assigned to one of seven categories:

- *Functional Experience* — Core feature bugs, missing functionality, and usability of primary application flows.
- *System & Device Compatibility* — Issues specific to certain device models, screen sizes, or HarmonyOS API versions.
- *Stability & Performance* — Crashes, freezes, memory leaks, battery drain, and response lag.
- *UI & Interaction* — Visual design complaints, layout problems, navigation difficulties, and animation issues.
- *Operations & Policies* — Problems with ads, in-app purchases, account bans, content moderation, or pricing.
- *Appreciation* — General praise and positive feedback not tied to a specific feature.
- *Content Ecosystem* — Issues about content quality, availability, recommendation accuracy, or community management.

For each classified issue, the LLM also generates an **Actionable Suggestion** — a concrete recommendation that a developer could act on (e.g., “Add support for foldable screen aspect ratios in the video player”). For positive feedback, the suggestion field is set to *None*.

The system prompt instructs the LLM to output strictly valid JSON conforming to a pre-defined schema, enabling programmatic validation and error recovery. If the LLM response fails JSON parsing, an automatic repair pass is attempted before logging the batch as an error.

3.3 Topic Clustering

After classification, we group related pain points into coherent topic clusters. This is non-trivial because multiple comments may describe the same issue using superficially different language. For example, “连朋友资料都没，这算什么社交软件” and “好友来源、好友共同群聊、添加好友时间等信息都没有显示” use entirely different wording but both complain about the same missing feature: friend profile information. A statistical clustering method based on word embeddings would likely separate them into unrelated clusters due to near-zero lexical overlap; our solution uses the LLM directly as the clustering engine, which can reason that both comments target the same root cause—insufficient friend information display.

Clustering Rules. The System Prompt injects explicit rules guiding the LLM: merge items describing the same feature with different wording (e.g., synonyms for “light sensor”), ignore superficial differences such as device model or OS version when the root cause is identical, split comments that contain multiple independent issues, and guarantee that every input item appears in exactly one cluster. Each output cluster includes a `cluster_name` (4–12 Chinese characters) and a `canonical_issue` (a one-sentence description of the root cause).

Two-Stage Pipeline. Clustering proceeds in two stages:

1. **Stage 1 — Intra-bucket clustering.** Classified items are partitioned into up to 21 homogeneous (dimension, sentiment) buckets. Within each bucket, items are deduplicated and sent to the LLM in batches of up to 18. The LLM groups them into root-cause clusters, each containing a `cluster_name` (4–12 Chinese characters), a `canonical_issue` (one-sentence root-cause description), `size` (number of member items), and `member_ids` (references back to all source comment IDs).
2. **Stage 2 — Intra-bucket refinement.** Stage 1 splits large buckets into independent batches, which may independently produce semantically overlapping clusters. Stage 2 collects all clusters within each bucket and re-feeds them to the LLM for cross-batch deduplication and label standardization, producing a consolidated set of professionally named clusters with merged member assignments.

Error Recovery. To handle non-deterministic LLM outputs, a three-tier repair pipeline is applied: automatic deduplication removes duplicate item assignments and hallucinated IDs; token-overlap reassignment matches unassigned items to existing clusters via word overlap; and a secondary LLM repair call is invoked as a last resort. Any item that cannot be assigned is placed into a singleton cluster to prevent data loss. The pipeline supports `-resume` to persist intermediate results after each batch and continue from interruptions.

3.4 Web Dashboard

A lightweight web dashboard (Figure 2) provides an intuitive interface for browsing the analysis results:

- **Overview tab** — Aggregate statistics (total comments, classified count, sentiment distribution) rendered as donut and bar charts using inline SVG.
- **Comments tab** — Paginated, searchable list of raw comments with username, rating, date, location, and device metadata.
- **Analysis tab** — Dimension-filtered view showing each extracted pain point alongside its actionable suggestion and original comment context.
- **Clusters tab** — Topic clusters organized by dimension and sentiment, with expandable sample members.

The frontend is built with vanilla JavaScript (no framework dependencies) and communicates with a Python HTTP server (`server.py`) that serves static files and exposes RESTful API endpoints (`/api/apps`, `/api/apps/<pkg>/comments`, `/api/apps/<pkg>/analysis`, `/api/apps/<pkg>/clusters`).

AGCommentSpy — Comment Analysis Dashboard	
<i>Statistics</i>	Total comments, classified count, sentiment breakdown
<i>Charts</i>	Donut chart (sentiment), bar chart (dimension distribution)
<i>Comments</i>	Paginated raw comments with full-text search
<i>Analysis</i>	Per-dimension pain points with LLM-generated suggestions
<i>Clusters</i>	Topic clusters with canonical issue descriptions

Figure 2: Structure of the web dashboard.

4 Implementation

4.1 Technology Stack

Table 1 summarizes the key technologies used in each component.

Table 1: Technology stack by component.

Component	Technology	Purpose
Device automation	hmdriver2, HDC	UI interaction on HarmonyOS
XML/UI parsing	lxml, XPath	Extract comment data from UI hierarchy
LLM client	OpenAI-compatible API	Comment classification & clustering
Batch orchestration	Python subprocess	Sequential multi-app scraping
Backend server	Python http.server	REST API for dashboard data
Frontend	Vanilla JS, SVG, CSS	Interactive visualization
Data storage	JSON files (per-app directories)	Raw, classified, and clustered data
Language	Python 3.13	All backend logic
Target platform	HarmonyOS (OpenHarmony 4/5)	Device under automation

4.2 LLM Configuration

The LLM client (`llm_client.py`) communicates with any OpenAI-compatible API endpoint. Key design decisions include:

- **JSON mode.** The API is called with `response_format: {type: "json_object"}` to enforce structured output. If the endpoint does not support this parameter, the call is retried without it.
- **Batch processing.** Comments are sent in batches (configurable batch size, default 10) to limit prompt length and improve classification quality. Each batch is processed independently.
- **Error recovery.** A three-tier recovery strategy handles malformed outputs: (1) JSON extraction from markdown code fences, (2) heuristic brace-based extraction, and (3) a secondary LLM repair call with a dedicated repair prompt.
- **Idempotent output.** Each classified item retains the original comment ID (e.g., `com.tencent.wechat_42`), enabling traceability from dashboard back to raw comments.

4.3 Deduplication and Normalization

During clustering, identical issues from different source comments are deduplicated by normalizing text (lowercasing, whitespace collapsing, zero-width character stripping) and comparing the (dimension, sentiment, normalized pain point) triple, preventing semantically identical items from being counted multiple times.

4.4 Project Structure

Listing 2: Project directory layout.

```
1 project/
2   app.json           # Target app definitions
3   config.py          # Device ID, screen bounds, XPath constants
4   main.py            # Single-app scraper entry point
5   comments.py        # Comment parsing & scrolling logic
6   analyze_comments.py # LLM classification pipeline
7   llm_client.py       # OpenAI-compatible API client
8   cluster_and_label.py # Item collection & text normalization
9   server.py           # HTTP server with REST API
10  run_batch.py        # Batch orchestrator
11  patch_dates.py       # Date normalization utility
12  layout_output.py     # Layout XML/JSON dump helpers
13  xml_utils.py         # JSON-to-XML conversion & XMLElement wrapper
14  frontend/
15    index.html         # Dashboard entry point
16    css/style.css      # Dashboard styles
17    js/api.js          # API client
18    js/charts.js       # SVG chart rendering
19    js/app.js          # Dashboard application logic
20  scripts/
21    aggregate_classified.py # Aggregate raw LLM responses
22    llm_cluster_and_label.py # Stage-1 clustering
23    refine_and_merge_topics.py # Stage-2 merging & label standardization
24  com.<package>/        # Per-app output directories
25    app_info.json
26    comments.json
27    classified_comments.json
28    classified_comments.raw.json
29    clusters_preview_llm.json
```

5 Dataset

The scraping pipeline was executed against a diverse set of HarmonyOS applications listed in Table 2, spanning the following categories:

- **Social & Communication:** WeChat, QQ, Douyin (TikTok), Kuaishou, Xiaohongshu (RED), Weibo, DingTalk, Tencent Meeting
- **E-commerce:** Taobao, JD.com, Pinduoduo, Xianyu (Idle Fish), Vipshop, Dewu
- **Finance:** Alipay, ABC (Agricultural Bank of China), CCB, ICBC, Bank of China
- **Entertainment & Media:** Bilibili, iQiyi, Youku, Tencent Video, QQ Music, Kugou Music, Ximalaya, Tomato Novel, Red Fruit Short Drama
- **Tools & Services:** Amap (Gaode), Baidu Map, Baidu Search, Baidu Netdisk, Quark Browser, UC Browser, QQ Browser, WPS Office
- **Lifestyle & Travel:** Meituan, Dianping, Ctrip, Qunar, DiDi, 12306 Railway
- **Games:** Honor of Kings, Game for Peace, Teamfight Tactics, Subway Surfers, Happy Match
- **AI & Creativity:** Doubao (ByteDance AI assistant), Jianying (CapCut)

Table 2: Partial list of scraped applications (*full list in app.json*).

App Name	Package Name	Category
WeChat	com.tencent.wechat	Social
Douyin	com.ss.hm.ugc.aweme	Social
Alipay	com.alipay.mobile.client	Finance
Taobao	com.taobao.taobao4hmos	E-commerce
Meituan	com.sankuai.hmeituan	Lifestyle
Bilibili	yytx.danmaku.bili	Entertainment
Amap	com.amap.hmapp	Navigation
Honor of Kings	com.tencent.tmgp.sgame.hw	Gaming

For each application, up to 500 comment swipes were performed. The total raw dataset comprises over 10,000 individual comments, though the actual count per app varies depending on the volume of reviews available on the AppGallery.

6 Qualitative Discussion

6.1 Classification and Taxonomy Design

The seven-dimension taxonomy was designed to balance granularity with actionability:

- Dimensions like *Stability & Performance* and *System & Device Compatibility* target issues that require engineering investigation.
- *UI & Interaction* and *Functional Experience* capture user-facing quality concerns.
- *Operations & Policies* and *Content Ecosystem* cover business and content-side concerns.
- *Appreciation* serves as a catch-all for positive feedback that does not require developer action.

By requiring the LLM to output `Actionable_Suggestion` for every negative entry, the pipeline ensures that each finding is paired with a concrete improvement proposal. Positive and neutral entries do not generate suggestions, preventing noise in the developer-facing output.

6.2 Clustering Approach Comparison

Section 3.3 describes our final LLM-as-clusterer design. This section discusses the design rationale behind that choice.

Our initial clustering prototype followed a conventional NLP workflow: pain-point texts were embedded with Sentence-BERT [6], clustered via KMeans or DBSCAN, and each resulting cluster was summarized by the LLM into a label. We observed two fundamental problems with this approach:

1. **Surface-form blindness.** Revisiting the friend-information example from Section 3.3: the phrases “连朋友资料都没” and “好友来源、好友共同群聊等都没有” share almost no common words, so their embedding vectors would be far apart and assigned to different clusters—despite both targeting the identical root cause of insufficient friend information display. Word embeddings capture lexical similarity but cannot detect root-cause equivalence.
2. **Short-text noise.** Most pain-point items are 10–30 Chinese characters, yielding sparse, high-dimensional vectors that make unsupervised clustering boundaries unreliable.

These observations motivated the LLM-as-clusterer approach, where the LLM directly partitions items without an intermediate embedding step. The System Prompt injects domain rules—merge synonyms, ignore device-model differences, split multi-issue items, guarantee coverage—which allow the model to reason about root causes rather than lexical surface forms.

The two-stage decomposition addresses a fundamental tension: the full set of classified items exceeds a single LLM prompt, but splitting items arbitrarily would break semantic dependencies across batches. We resolve this by first partitioning items by (dimension, sentiment), so each batch is semantically homogeneous (e.g., all complaints about UI and interaction). This focus increases the chance that

related items co-occur in the same batch and are correctly merged. However, batching introduces a side effect—different batches within the same bucket may independently discover the same root cause. Stage 2 reconciles this by re-feeding the much smaller set of cluster summaries to the LLM in one call for cross-batch deduplication and label standardization. In essence, the two-stage design decomposes a large clustering problem into focused sub-problems with a lightweight reconciliation step.

Our qualitative inspection indicates that this approach produces substantially more coherent groupings than the embedding-based alternative.

6.3 Robustness Mechanisms

The LLM-as-clusterer approach introduces a practical challenge: LLM outputs are non-deterministic. Across our dataset, we observed recurring failure modes—duplicate item assignments across multiple clusters, missing items not assigned to any cluster, and hallucinated item IDs. The three-tier repair pipeline described in Section 3.3 was developed to handle these issues progressively:

1. **Automatic deduplication** resolves duplicate assignments by retaining the first occurrence.
2. **Token-overlap reassignment** matches unassigned items to existing clusters by computing simple word overlap between the item text and cluster descriptors.
3. **LLM re-repair** is invoked only as a last resort, re-feeding the original output with validation errors to the model.

This layered design reflects a practical principle when building LLM-integrated pipelines: deterministic programmatic repair should be exhausted before resorting to additional LLM calls, which are both expensive and themselves fallible. The singleton fallback ensures that no item is silently discarded: even if all repair steps fail, the item remains visible in the dashboard as a standalone entry.

6.4 Limitations

Several limitations should be acknowledged:

1. **Scraping reliability.** UI automation is inherently fragile. Changes to the AppGallery UI layout (XPath selectors) can break the scraper and require manual selector updates in `config.py`.
2. **LLM dependence.** The classification and clustering quality depends on the underlying LLM. Different models or API providers may produce varying quality, and prompt engineering is model-specific.
3. **Single-language focus.** The current system prompt and taxonomy are designed for Chinese-language comments. Extending to multilingual analysis would require prompt adaptation.
4. **No ground-truth evaluation.** We did not conduct a formal human-annotated evaluation of classification or clustering accuracy. Future work should include a manually labeled subset to enable quantitative measurement of precision and recall.
5. **Device requirement.** The scraper requires a physical HarmonyOS device connected via HDC, limiting scalability compared to API-based collection methods.

7 Conclusion and Future Work

This paper presented AGCOMMENTSPY, a complete pipeline for collecting and analyzing user comments from the Huawei AppGallery on HarmonyOS. The system demonstrates that combining UI automation with LLM-based natural language understanding can effectively transform unstructured app store reviews into actionable developer insights. The web dashboard provides an accessible interface for browsing sentiment distributions, dimension breakdowns, and topic clusters across a diverse set of real-world applications.

Several directions for future work are apparent:

1. **Automated labeling for evaluation.** Build a human-annotated gold standard subset to quantitatively measure classification precision and recall.

2. **Multilingual support.** Extend the taxonomy and prompts to support English and other languages present in international AppGallery listings.
3. **LLM comparison.** Benchmark multiple LLM providers and model sizes on the same dataset to identify cost-quality trade-offs.
4. **Incremental scraping.** Support scraping only new comments since the last run, reducing processing overhead for ongoing monitoring.
5. **Automated issue reporting.** Integrate with developer workflow tools (e.g., GitHub Issues, Jira) to automatically file actionable findings from the analysis pipeline.

Acknowledgments

This project builds on the open-source `hmdriver2` library [7] for HarmonyOS device interaction. We thank the contributors of that project for enabling programmatic access to HarmonyOS devices.

References

- [1] Huawei Technologies Co., Ltd. *HarmonyOS Developer Guide*. <https://developer.huawei.com/consumer/en/harmonyos/>, 2025.
- [2] Huawei Technologies Co., Ltd. *AppGallery Connect*. <https://developer.huawei.com/consumer/en/appgallery/>, 2025.
- [3] D. Pagano and W. Maalej. User feedback in the appstore: An empirical study. In *Proc. 21st IEEE International Requirements Engineering Conference (RE)*, pp. 125–134, 2013.
- [4] W. Martin, F. Sarro, Y. Jia, Y. Zhang, and M. Harman. A survey of app store analysis for software engineering. *IEEE Transactions on Software Engineering*, 43(9):817–847, 2017.
- [5] Z. Zou, Y. Zhao, Y. Liu, et al. Large language models for app review analysis: An exploratory study. In *Proc. ACM International Conference on the Foundations of Software Engineering (FSE)*, 2024.
- [6] N. Reimers and I. Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019. arXiv:1908.10084.
- [7] CodeMatrix. `hmdriver2`: Python client for HarmonyOS HDC. <https://github.com/codematrixer/hmdriver2>, 2025.
- [8] Google LLC. *UI Automator — Android Developers*. <https://developer.android.com/training/testing/other-components/ui-automator>, 2025.
- [9] Appium Contributors. *Appium: Automation for Apps*. <https://appium.io/>, 2025.